

. What is OOP?

OOP organizes a program around **objects** rather than just functions and variables.

An object combines:

DATA + BEHAVIOR

For example, a **BankAccount** object can contain:

Data:

- account_number
- owner
- balance

Behavior:

- deposit()
- withdraw()
- check_balance()

In Python:

```
class BankAccount:
```

```
    def deposit(self):  
        pass
```

```
    def withdraw(self):  
        pass
```

2. Class vs Object

Class

A class is a **blueprint**.

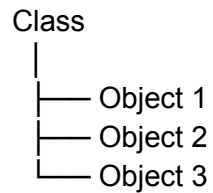
```
class Student:  
    pass
```

Object

An object is an **instance of a class**.

```
student1 = Student()  
student2 = Student()
```

Think of it as:



Example:

```
class Car:  
    pass
```

```
car1 = Car()  
car2 = Car()
```

```
print(type(car1))
```

Output:

```
<class '__main__.Car'>
```

3. Creating Attributes

Attributes represent the **data/state** of an object.

```
class Student:  
    pass
```

```
student = Student()
```

```
student.name = "Rahul"  
student.age = 22  
student.marks = 85
```

```
print(student.name)
print(student.age)
print(student.marks)
```

Output:

```
Rahul
22
85
```

However, this isn't usually the preferred way to initialize objects.

That's where `__init__()` comes in.

4. The Constructor — `__init__()`

class Student:

```
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks
```

```
student = Student("Rahul", 22, 85)
```

```
print(student.name)
print(student.age)
print(student.marks)
```

When we execute:

```
student = Student("Rahul", 22, 85)
```

Python automatically calls:

```
__init__()
```

5. Understanding **self**

This is one of the most important OOP concepts for Python beginners.

```
class Student:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def display(self):  
        print(self.name)
```

When we write:

```
student = Student("Rahul")
```

```
student.display()
```

Python effectively passes the object into the method:

```
Student.display(student)
```

Therefore:

```
self
```

refers to the current object.

For example:

```
student1 = Student("Rahul")  
student2 = Student("Priya")
```

Inside:

```
student1.display()
```

self refers to **student1**.

Inside:

```
student2.display()
```

self refers to **student2**.

6. Instance Variables

Instance variables belong to a specific object.

```
class Employee:
```

```
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
e1 = Employee("Rahul", 50000)
e2 = Employee("Priya", 60000)
```

```
print(e1.name)
print(e2.name)
```

Conceptually:

```
e1
├── name = Rahul
└── salary = 50000
```

```
e2
├── name = Priya
└── salary = 60000
```

7. Instance Methods

Methods operate on object data.

```
class Employee:
```

```
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
    def display(self):
        print(self.name)
        print(self.salary)
```

```
employee = Employee("Rahul", 50000)
```

```
employee.display()
```

8. Instance Attributes vs Local Variables

Consider:

```
class Student:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def display(self):
```

```
        message = "Hello"
```

```
        print(self.name)
```

```
        print(message)
```

Here:

`self.name` → instance variable

`message` → local variable

`self.name` belongs to the object.

`message` exists only while `display()` executes.

9. Class Variables

A class variable is shared by objects.

```
class Employee:
```

```
    company = "ABC Technologies"
```

```
    def __init__(self, name):
```

```
self.name = name
```

```
e1 = Employee("Rahul")  
e2 = Employee("Priya")
```

```
print(e1.company)  
print(e2.company)
```

Both objects access:

ABC Technologies

10. Instance Variable vs Class Variable

class Employee:

```
    company = "ABC"
```

```
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary
```

Here:

company → class variable

name → instance variable

salary → instance variable

Use class variables when the value logically belongs to the **class as a whole**.

11. Class Methods

A class method works with the class rather than a particular object.

Use:

@classmethod

and `cls`.

```
class Employee:
```

```
    company = "ABC"
```

```
    @classmethod
```

```
    def change_company(cls, new_company):
```

```
        cls.company = new_company
```

```
Employee.change_company("XYZ")
```

```
print(Employee.company)
```

Output:

XYZ

12. Static Methods

A static method doesn't require `self` or `cls`.

```
class Calculator:
```

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

```
print(Calculator.add(10, 20))
```

Output:

30

Use a static method when functionality logically belongs to the class but doesn't need object/class state.

13. Three Main Method Types

Method	First parameter	Decorator
Instance method	<code>self</code>	None
Class method	<code>cls</code>	<code>@classmethod</code>
Static method	None	<code>@staticmethod</code>

Example:

class Demo:

```
def instance_method(self):  
    pass
```

```
@classmethod  
def class_method(cls):  
    pass
```

```
@staticmethod  
def static_method():  
    pass
```

14. Encapsulation

Encapsulation means keeping related data and functionality together and controlling access to internal state.

Example:

class BankAccount:

```
def __init__(self, balance):  
    self.__balance = balance
```

```
def deposit(self, amount):
```

```
if amount > 0:  
    self.__balance += amount
```

```
def get_balance(self):  
    return self.__balance
```

Usage:

```
account = BankAccount(10000)
```

```
account.deposit(5000)
```

```
print(account.get_balance())
```

Output:

```
15000
```

15. Public, Protected and Private

Python uses naming conventions.

Public

```
self.name
```

Accessible normally.

Protected convention

```
self._salary
```

The single underscore means:

This is intended for internal/subclass use.

Python doesn't strictly prevent access.

Private/name-mangled

```
self.__salary
```

Python performs name mangling.

Example:

```
class Employee:
```

```
    def __init__(self):  
        self.__salary = 50000
```

The attribute is internally represented approximately as:

```
_Employee__salary
```

16. Getters and Setters

Instead of allowing unrestricted modification:

```
employee.salary = -50000
```

we can control it.

```
class Employee:
```

```
    def __init__(self, salary):  
        self.__salary = salary
```

```
    def get_salary(self):  
        return self.__salary
```

```
    def set_salary(self, salary):
```

```
        if salary >= 0:  
            self.__salary = salary  
        else:  
            print("Invalid salary")
```

```
employee = Employee(50000)
```

```
print(employee.get_salary())
```

```
employee.set_salary(60000)
```

```
print(employee.get_salary())
```

17. @property

Python provides a cleaner approach.

```
class Employee:

    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, value):

        if value < 0:
            raise ValueError("Salary cannot be negative")

        self.__salary = value
```

Usage:

```
employee = Employee(50000)

print(employee.salary)

employee.salary = 60000

print(employee.salary)
```

The syntax looks like an attribute, but internally methods are being called.

18. Inheritance

Inheritance allows a class to derive functionality from another class.

```
class Animal:
```

```
    def eat(self):
```

```
print("Eating")
```

```
class Dog(Animal):
```

```
    def bark(self):  
        print("Barking")
```

Usage:

```
dog = Dog()
```

```
dog.eat()  
dog.bark()
```

Output:

```
Eating  
Barking
```

19. Parent and Child Classes

Terminology:

Animal → Parent / Base / Superclass

Dog → Child / Derived / Subclass

Example:

```
class Animal:
```

```
    def eat(self):  
        print("Animal eats")
```

```
class Dog(Animal):
```

```
    def bark(self):  
        print("Dog barks")
```

20. Method Overriding

A child can redefine a parent's method.

```
class Animal:
```

```
    def sound(self):  
        print("Animal sound")
```

```
class Dog(Animal):
```

```
    def sound(self):  
        print("Bark")
```

```
dog = Dog()
```

```
dog.sound()
```

Output:

Bark

The child implementation overrides the inherited implementation.

21. `super()`

`super()` is used to access the parent implementation.

```
class Employee:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class Manager(Employee):
```

```
    def __init__(self, name, department):
```

```
super().__init__(name)
```

```
self.department = department
```

Now:

```
manager = Manager("Rahul", "IT")
```

```
print(manager.name)
```

```
print(manager.department)
```

22. Multilevel Inheritance

Person

↓

Employee

↓

Manager

```
class Person:
```

```
    def introduce(self):  
        print("I am a person")
```

```
class Employee(Person):
```

```
    def work(self):  
        print("I work")
```

```
class Manager(Employee):
```

```
    def manage(self):  
        print("I manage a team")
```

```
manager = Manager()
```

```
manager.introduce()
```

```
manager.work()
```

```
manager.manage()
```

23. Multiple Inheritance

A class can inherit from multiple classes.

```
class Father:
```

```
    def father_property(self):  
        print("Father property")
```

```
class Mother:
```

```
    def mother_property(self):  
        print("Mother property")
```

```
class Child(Father, Mother):  
    pass
```

```
child = Child()
```

```
child.father_property()  
child.mother_property()
```

24. Method Resolution Order — MRO

When multiple inheritance exists, Python needs to determine which method to use.

Python uses **MRO — Method Resolution Order**.

```
class A:  
    pass
```

```
class B(A):  
    pass
```

```
class C(A):  
    pass
```



```
class D(B, C):  
    pass
```

```
print(D.mro())
```

You'll see a sequence similar to:

$D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$

You can also use:

```
print(D.__mro__)
```

25. Polymorphism

Polymorphism means:

The same interface can produce different behavior.

```
class Dog:
```

```
    def sound(self):  
        return "Bark"
```

```
class Cat:
```

```
    def sound(self):  
        return "Meow"
```

```
animals = [Dog(), Cat()]
```

```
for animal in animals:  
    print(animal.sound())
```

Output:

Bark
Meow

The code:

```
animal.sound()
```

works with different object types.

26. Duck Typing

Python follows a strong duck-typing philosophy.

```
class Dog:
```

```
    def speak(self):  
        print("Bark")
```

```
class Person:
```

```
    def speak(self):  
        print("Hello")
```

```
def speak(obj):  
    obj.speak()
```

```
speak(Dog())  
speak(Person())
```

Python doesn't require both objects to inherit from the same class.

It simply requires:

```
obj.speak()
```

to work.

27. Abstraction

Abstraction hides implementation details and defines an expected interface.

Use:

```
from abc import ABC, abstractmethod
```

Example:

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):
```

```
    @abstractmethod
    def pay(self, amount):
        pass
```

A concrete implementation:

```
class CreditCardPayment(Payment):
```

```
    def pay(self, amount):
        print(f"Paid ₹{amount} using credit card")
```

Another:

```
class UPIPayment(Payment):
```

```
    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI")
```

Usage:

```
payments = [
    CreditCardPayment(),
    UPIPayment()
]
```

```
for payment in payments:
    payment.pay(1000)
```

28. Composition

Composition means building one object using other objects.

Instead of:

Car IS-A Engine

we say:

Car HAS-A Engine

Example:

```
class Engine:
```

```
    def start(self):  
        print("Engine started")
```

```
class Car:
```

```
    def __init__(self):  
        self.engine = Engine()
```

```
    def start(self):  
        self.engine.start()  
        print("Car started")
```

Usage:

```
car = Car()
```

```
car.start()
```

This is **composition**.

29. Inheritance vs Composition

Inheritance

Use when there is an **IS-A** relationship.

Dog IS-A Animal

Manager IS-A Employee

Composition

Use when there is a **HAS-A** relationship.

Car HAS-A Engine

Computer HAS-A CPU

House HAS-A Room

In large software systems, composition is often preferable to deep inheritance hierarchies.

30. Magic Methods

Magic methods are also called **dunder methods** because they use double underscores.

Examples:

```
__init__  
__str__  
__repr__  
__len__  
__eq__  
__lt__  
__gt__  
__add__  
__sub__  
__mul__
```

31. `__str__()`

class Student:

```
    def __init__(self, name):  
        self.name = name  
  
    def __str__(self):  
        return f"Student: {self.name}"
```

```
student = Student("Rahul")
```

```
print(student)
```

Output:

Student: Rahul

Without `__str__()`, Python uses the default object representation.

32. `__repr__()`

`__repr__()` is intended to provide a useful developer-oriented representation.

class Student:

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def __repr__(self):
```

```
        return f"Student(name={self.name!r}, marks={self.marks})"
```

Now:

```
student = Student("Rahul", 85)
```

```
print(repr(student))
```

Output:

Student(name='Rahul', marks=85)

33. `__len__()`

class Team:

```
    def __init__(self, players):
```

```
        self.players = players

    def __len__(self):
        return len(self.players)

team = Team(["A", "B", "C"])

print(len(team))
```

Output:

3

34. `__eq__()`

Controls `==`.

```
class Student:

    def __init__(self, student_id):
        self.student_id = student_id

    def __eq__(self, other):
        return self.student_id == other.student_id

s1 = Student(101)
s2 = Student(101)

print(s1 == s2)
```

Output:

True

35. Operator Overloading

We can define how operators behave with our objects.

```
class Point:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(
            self.x + other.x,
            self.y + other.y
        )

    def __str__(self):
        return f"({self.x}, {self.y})"
```

```
p1 = Point(2, 3)
p2 = Point(4, 5)
```

```
p3 = p1 + p2
```

```
print(p3)
```

Output:

```
(6, 8)
```

Python translates:

```
p1 + p2
```

into something conceptually similar to:

```
p1.__add__(p2)
```

36. Comparison Magic Methods

You can implement:

```
__eq__  # ==
__ne__  # !=
__lt__  # <
__le__  # <=
```



```
__gt__ # >
__ge__ # >=
```

Example:

```
class Student:
```

```
    def __init__(self, marks):
        self.marks = marks

    def __gt__(self, other):
        return self.marks > other.marks
```

```
s1 = Student(90)
s2 = Student(75)
```

```
print(s1 > s2)
```

Output:

True

37. `__add__`, `__sub__`, `__mul__`

```
class Number:
```

```
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __sub__(self, other):
        return Number(self.value - other.value)

    def __mul__(self, other):
        return Number(self.value * other.value)

    def __str__(self):
        return str(self.value)
```

Usage:

```
a = Number(10)
b = Number(5)
```

```
print(a + b)
print(a - b)
print(a * b)
```

38. `__call__()`

An object can be made callable.

```
class Greeter:
```

```
    def __call__(self, name):
        print(f"Hello {name}")
```

```
greet = Greeter()
```

```
greet("Rahul")
```

Output:

Hello Rahul

Normally:

```
greet()
```

would fail because the object isn't callable.

With `__call__()`, it works.

39. `__contains__()`

Controls the `in` operator.

```
class Team:
```

```
def __init__(self, players):
    self.players = players

def __contains__(self, player):
    return player in self.players
```

```
team = Team(["Rahul", "Priya", "Arun"])
```

```
print("Rahul" in team)
```

Output:

```
True
```

40. Iterators in Classes

We can make our objects iterable.

```
class Numbers:
```

```
    def __init__(self, numbers):
        self.numbers = numbers
```

```
    def __iter__(self):
        return iter(self.numbers)
```

```
numbers = Numbers([10, 20, 30])
```

```
for number in numbers:
    print(number)
```

41. Dataclasses

For classes primarily storing data, Python provides [dataclasses](#).

```
from dataclasses import dataclass
```

```
@dataclass
class Student:
    name: str
    age: int
    marks: float
```

```
student = Student("Rahul", 22, 85)
```

```
print(student)
```

Output resembles:

```
Student(name='Rahul', age=22, marks=85)
```

Dataclasses automatically provide useful methods such as initialization and representation.

42. Type Hints with Classes

Modern Python code commonly uses type hints.

```
class Student:

    def __init__(self, name: str, age: int, marks: float):
        self.name = name
        self.age = age
        self.marks = marks

    def result(self) -> str:

        if self.marks >= 40:
            return "Pass"

        return "Fail"
```

Type hints improve readability and tooling.

They aren't, by themselves, strict runtime validation.

43. Abstract Class + Polymorphism

This is a very common real-world design.

```
from abc import ABC, abstractmethod
```

```
class Employee(ABC):  
  
    def __init__(self, name):  
        self.name = name  
  
    @abstractmethod  
    def calculate_salary(self):  
        pass
```

Full-time employee:

```
class FullTimeEmployee(Employee):  
  
    def __init__(self, name, salary):  
        super().__init__(name)  
        self.salary = salary  
  
    def calculate_salary(self):  
        return self.salary
```

Part-time employee:

```
class PartTimeEmployee(Employee):  
  
    def __init__(self, name, hours, rate):  
        super().__init__(name)  
        self.hours = hours  
        self.rate = rate  
  
    def calculate_salary(self):  
        return self.hours * self.rate
```

Now:

```
employees = [  
    FullTimeEmployee("Rahul", 60000),  
    PartTimeEmployee("Priya", 80, 500)
```

```
]
```

```
for employee in employees:  
    print(employee.name, employee.calculate_salary())
```

This demonstrates:

Abstraction
+
Inheritance
+
Polymorphism
+
Encapsulation

44. SOLID Principles

For professional OOP, learn the **SOLID** principles.

Principle	Meaning
S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

S — Single Responsibility

A class should have one primary responsibility.

Bad:

```
class Employee:  
  
    def calculate_salary(self):  
        pass
```

```
def save_to_database(self):  
    pass
```

```
def generate_pdf(self):  
    pass
```

```
def send_email(self):  
    pass
```

Better:

Employee
SalaryCalculator
EmployeeRepository
ReportGenerator
EmailService

45. Dependency Injection

Instead of a class creating its dependency internally:

class Car:

```
def __init__(self):  
    self.engine = Engine()
```

we can inject it:

class Car:

```
def __init__(self, engine):  
    self.engine = engine
```

Now:

```
engine = Engine()
```

```
car = Car(engine)
```

This makes code easier to test and maintain.

46. A Complete OOP Project

Let's combine the concepts.

Student Management System

```
from abc import ABC, abstractmethod
import numpy as np
```

```
class Person(ABC):
```

```
    def __init__(self, name):
        self.name = name
```

```
    @abstractmethod
    def display(self):
        pass
```

```
class Student(Person):
```

```
    school = "ABC Academy"
```

```
    def __init__(self, student_id, name, marks):
```

```
        super().__init__(name)
```

```
        self.student_id = student_id
        self.marks = np.array(marks)
```

```
    def average(self):
        return np.mean(self.marks)
```

```
    def highest(self):
        return np.max(self.marks)
```

```
    def lowest(self):
        return np.min(self.marks)
```

```
    def total(self):
        return np.sum(self.marks)
```



```

def grade(self):

    avg = self.average()

    if avg >= 90:
        return "A"

    elif avg >= 75:
        return "B"

    elif avg >= 60:
        return "C"

    elif avg >= 40:
        return "D"

    return "F"

def display(self):

    print(f"ID: {self.student_id}")
    print(f"Name: {self.name}")
    print(f"Marks: {self.marks}")
    print(f"Total: {self.total()}")
    print(f"Average: {self.average():.2f}")
    print(f"Highest: {self.highest()}")
    print(f"Lowest: {self.lowest()}")
    print(f"Grade: {self.grade()}")

def __str__(self):
    return (
        f"{self.student_id} - "
        f"{self.name} - "
        f"Average: {self.average():.2f}"
    )

```

Create objects:

```

s1 = Student(
    101,
    "Rahul",
    [80, 75, 90, 85, 88]
)

s2 = Student(

```

```
102,  
"Priya",  
[92, 95, 88, 90, 94]  
)  
  
s1.display()  
  
print()  
  
s2.display()
```

47. Student Management Class

Now create a class that manages multiple students.

```
class StudentManagementSystem:  
  
    def __init__(self):  
        self.students = []  
  
    def add_student(self, student):  
        self.students.append(student)  
  
    def display_students(self):  
  
        for student in self.students:  
            print(student)  
  
    def topper(self):  
  
        return max(  
            self.students,  
            key=lambda student: student.average()  
        )  
  
    def class_average(self):  
  
        averages = [  
            student.average()  
            for student in self.students  
        ]
```

```
return np.mean(averages)
```

Usage:

```
system = StudentManagementSystem()
```

```
system.add_student(s1)
```

```
system.add_student(s2)
```

```
system.display_students()
```

```
topper = system.topper()
```

```
print("Topper:", topper.name)
```

```
print(  
    "Class Average:",  
    system.class_average()  
)
```

This is a much more realistic example of why OOP is useful.

48. OOP Mental Model

When designing a program, ask:

Step 1 — What are the objects?

For a banking application:

Customer

Account

Transaction

Bank

Step 2 — What data does each object have?

Customer

name

phone

address

Account
 account_number
 balance
 account_type

Step 3 — What can each object do?

Account
 deposit()
 withdraw()
 transfer()

Step 4 — What relationships exist?

Customer HAS-A Account

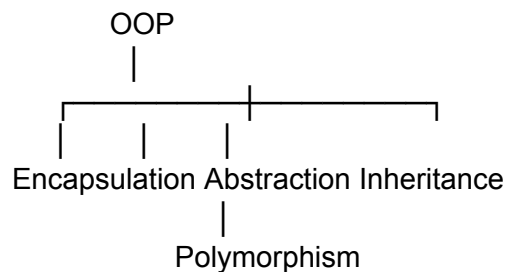
SavingsAccount IS-A Account

CurrentAccount IS-A Account

This thinking process is more important than memorizing syntax.

49. The Four Pillars of OOP

The four concepts you should absolutely master are:



Encapsulation

Protect/manage object state.

Abstraction

Expose essential functionality while hiding implementation details.

Inheritance

Reuse/extend behavior from another class.

Polymorphism

Same interface, different implementations.

50. OOP Learning Roadmap

For trainees, I recommend learning in this order:

1. Classes
- ↓
2. Objects
- ↓
3. self
- ↓
4. __init__()
- ↓
5. Instance variables
- ↓
6. Instance methods
- ↓
7. Class variables
- ↓
8. Class methods
- ↓
9. Static methods
- ↓
10. Encapsulation
- ↓
11. Properties
- ↓
12. Inheritance
- ↓
13. super()
- ↓
14. Method overriding
- ↓
15. Polymorphism

- ↓
 - 16. Abstraction
 - ↓
 - 17. Composition
 - ↓
 - 18. Magic methods
 - ↓
 - 19. Dataclasses
 - ↓
 - 20. SOLID
 - ↓
 - 21. Design patterns
-

51. OOP Interview Questions

A trainee should be able to answer these without hesitation:

Beginner

1. What is OOP?
2. What is a class?
3. What is an object?
4. What is `self`?
5. What is `__init__()`?
6. What are instance variables?
7. What are class variables?
8. What is an instance method?
9. What is a class method?
10. What is a static method?

Intermediate

11. What is inheritance?
12. What is method overriding?
13. What is `super()`?
14. What is multiple inheritance?
15. What is MRO?
16. What is encapsulation?
17. What is abstraction?
18. What is polymorphism?
19. What is duck typing?

20. What is composition?

Advanced

21. What are magic methods?

22. Difference between `__str__()` and `__repr__()`?

23. What is operator overloading?

24. What is `@property`?

25. What is an abstract base class?

26. What are dataclasses?

27. What is dependency injection?

28. What are SOLID principles?

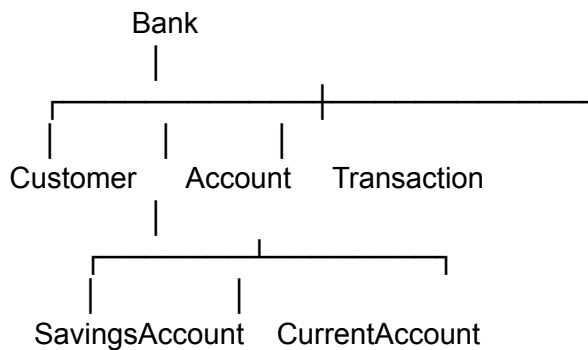
29. Inheritance vs composition?

30. How does Python handle multiple inheritance?

52. Final Practice Project

Build a **Banking Application** using OOP.

Your architecture should look like:



Implement:

Customer

- name
- customer_id
- accounts

Account

- account_number
- balance

```
|— deposit()
|— withdraw()
|— get_balance()
```

SavingsAccount
|— calculate_interest()

CurrentAccount
|— calculate_overdraft()

Transaction
|— amount
|— type
|— timestamp

The project should demonstrate:

- ✓ Classes & Objects
- ✓ Constructors
- ✓ Instance/Class/Static methods
- ✓ Encapsulation
- ✓ Properties
- ✓ Inheritance
- ✓ Method overriding
- ✓ super()
- ✓ Polymorphism
- ✓ Abstraction
- ✓ Composition
- ✓ Magic methods
- ✓ Exception handling
- ✓ Type hints